In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
pd.set_option("display.max_columns", None)
pd.set_option("display.max_rows", None)
```

In [ ]:
```
!pip install pandas numpy scikit-learn lightgbm xgboost torch
```

In [2]:
```
df = pd.read_csv(r"C:\Users\EIJEL\OneDrive - Bayer\Personal Data\Erik\2024\Consolidated\mldataInitialGermnew.c
dfq = pd.read_csv(r'C:\Users\EIJEL\OneDrive - Bayer\Personal Data\Erik\2024\pkl_new\cotton-dormancy\data\vw_co
dfqbulk = dfq[dfq.Stage==1]
dfqbulk = dfqbulk[['CT_Initial', 'CT_Initial_DT', 'WG_Initial_7Day_Cnt', 'WG_Initial_7Day_Cnt_DT', 'Bulk_Batch
```

```
C:\Users\EIJEL\AppData\Local\Temp\ipykernel_27372\325910757.py:2: DtypeWarning: Columns (13,16,24,132,133,134,135,136,137,138) have
mixed types. Specify dtype option on import or set low_memory=False.
  dfq = pd.read_csv(r'C:\Users\EIJEL\OneDrive - Bayer\Personal Data\Erik\2024\pkl_new\cotton-
dormancy\data\vw_cotton_lineage_and_quality_june_fg_all_cols_lotcreateddt.csv')
```

In [3]:
```
dfqbulk.head()
```

Out [3]:

	CT_Initial	CT_Initial_DT	WG_Initial_7Day_Cnt	WG_Initial_7Day_Cnt_DT	Bulk_Batch
0	NaN	NaN	NaN	NaN	922
1	74.25	2018-02-02	92.25	2018-02-05	A77U
73	65.75	2018-02-14	88.00	2018-02-14	A77V
95	74.25	2018-01-05	92.40	2018-01-05	A77Y
405	37.25	2018-02-20	56.50	2018-01-25	A786

In [3]:
```
df_truck_with_pa_feature_id = pd.read_csv(r"C:\Users\EIJEL\OneDrive - Bayer\Personal Data\Erik\2024\pkl_new\co
df_truck_with_pa_feature_id = df_truck_with_pa_feature_id.drop_duplicates(subset = ['pa_feature_id'])
cotton_with_truck = df.merge(df_truck_with_pa_feature_id[['pa_feature_id', 'bulk_batch', 'lb_received', 'inspc
```

In [4]:
```
sorted(cotton_with_truck.columns)
```

Out [4]:
```
['Bulk_Batch',
 'CT_Initial',
 'FUZZY_PRCS_ORD_NBR',
 'WG_Initial_7Day_Cnt',
 'accepted_modules',
 'ans_variety',
 'bales_per_module',
 'bolls_open_1',
 'bolls_open_1_date',
 'bolls_open_2',
 'bolls_open_2_date',
 'bolls_open_3',
 'bolls_open_3_date',
 'bolls_open_4',
 'bolls_open_4_date',
 'bulk_batch',
 'catch_rate',
 'centroid',
 'comments',
 'cotton_maczone',
 'country',
 'county',
 'crop',
 'cumulated_dd60',
 'cumulated_soil_moisture',
 'date',
 'dd60',
 'defol_to_harvest',
 'defol_to_harvest_average_dd60',
 'defol_to_harvest_average_heat',
 'defol_to_harvest_cum_dd60',
 'defol_to_harvest_cum_heat',
 'defoliation_1',
 'defoliation_2',
 'defoliation_3',
 'defoliation_date',
 'entityid',
 'field_status',
 'fuzzy_estimate',
 'grid_id',
 'group_id',
 'harvest_date',
 'harvest_date_fill',
 'inspct_lot_nbr',
 'irrigation',
 'irrigation_type',
 'lat',
 'lb_received',
 'lon',
 'maczone',
 'new_variety',
 'nodes_above_cracked_boll_1',
```

```
 'nodes_above_cracked_boll_1_date',
 'nodes_above_cracked_boll_2',
 'nodes_above_cracked_boll_2_date',
 'nodes_above_white_flower_1',
 'nodes_above_white_flower_1_date',
 'nodes_above_white_flower_2',
 'nodes_above_white_flower_2_date',
 'nodes_above_white_flower_3',
 'nodes_above_white_flower_3_date',
 'pa_area',
 'pa_feature_id',
 'pa_wkt',
 'pa_year',
 'pf_feature_id',
 'pf_name',
 'pf_type',
 'pf_wkt',
 'pf_year',
 'plant',
 'planted_surface',
 'planting_area_id',
 'planting_date',
 'plantname',
 'pp_day10_avg_dd60',
 'pp_day10_avg_soilmoisture',
 'pp_day10_cum_dd60',
 'pp_day10_cum_soilmoisture',
 'pp_day5_avg_dd60',
 'pp_day5_avg_soilmoisture',
 'pp_day5_cum_dd60',
 'pp_day5_cum_soilmoisture',
 'questiontext',
 'random_days',
 'random_days_high',
 'random_days_low',
 'rejected_modules',
 'rm',
 'row_spacing',
 'season',
 'season_length',
 'seeding_rate',
 'state',
 'thryvon',
 'total_modules',
 'var_bin',
 'variety',
 'yield',
 'zipcode']
```

In [6]:

```
sorted(cotton_with_truck.columns)
```

Out [6]:

```
['Bulk_Batch',
 'CT_Initial',
 'FUZZY_PRCS_ORD_NBR',
 'WG_Initial_7Day_Cnt',
 'accepted_modules',
 'ans_variety',
 'bales_per_module',
 'bolls_open_1',
 'bolls_open_1_date',
 'bolls_open_2',
 'bolls_open_2_date',
 'bolls_open_3',
 'bolls_open_3_date',
 'bolls_open_4',
 'bolls_open_4_date',
 'bulk_batch',
 'catch_rate',
 'centroid',
 'comments',
 'cotton_maczone',
 'country',
 'county',
 'crop',
 'cumulated_dd60',
 'cumulated_soil_moisture',
 'date',
 'dd60',
 'defol_to_harvest',
 'defol_to_harvest_average_dd60',
 'defol_to_harvest_average_heat',
 'defol_to_harvest_cum_dd60',
 'defol_to_harvest_cum_heat',
 'defoliation_1',
 'defoliation_2',
 'defoliation_3',
 'defoliation_date',
 'entityid',
 'field_status',
 'fuzzy_estimate',
 'grid_id',
 'group_id',
 'harvest_date',
 'harvest_date_fill',
 'inspct_lot_nbr',
 'irrigation',
 'irrigation_type',
 'lat',
 'lb_received',
 'lon',
 'maczone',
 'new_variety',
 'nodes_above_cracked_boll_1',
 'nodes_above_cracked_boll_1_date',
 'nodes_above_cracked_boll_2',
 'nodes_above_cracked_boll_2_date',
 'nodes_above_white_flower_1',
 'nodes_above_white_flower_1_date',
 'nodes_above_white_flower_2',
 'nodes_above_white_flower_2_date',
 'nodes_above_white_flower_3',
 'nodes_above_white_flower_3_date',
```

```
 'pa_area',
 'pa_feature_id',
 'pa_wkt',
 'pa_year',
 'pf_feature_id',
 'pf_name',
 'pf_type',
 'pf_wkt',
 'pf_year',
 'plant',
 'planted_surface',
 'planting_area_id',
 'planting_date',
 'plantname',
 'pp_day10_avg_dd60',
 'pp_day10_avg_soilmoisture',
 'pp_day10_cum_dd60',
 'pp_day10_cum_soilmoisture',
 'pp_day5_avg_dd60',
 'pp_day5_avg_soilmoisture',
 'pp_day5_cum_dd60',
 'pp_day5_cum_soilmoisture',
 'questiontext',
 'random_days',
 'random_days_high',
 'random_days_low',
 'rejected_modules',
 'rm',
 'row_spacing',
 'season',
 'season_length',
 'seeding_rate',
 'state',
 'thryvon',
 'total_modules',
 'var_bin',
 'variety',
 'yield',
 'zipcode']
```

In [7]:

```
# df_truck_with_pa_feature_id = \
# df_truck_with_pa_feature_id.sort_values(by = ['bulk_batch', 'lb_received'], \
#                                         ascending = [True, False]).drop_duplicates(subset = ['bulk_batch'], 
# df_single_batch = df.merge(df_truck_with_pa_feature_id[['pa_feature_id', 'bulk_batch']], on='pa_feature_id')
```

In [4]:

```
dffuzzyquery = pd.read_csv(r'C:\Users\EIJEL\OneDrive - Bayer\Personal Data\Erik\2024\pkl_new\cotton-dormancy\da
dffuzzyquery = dffuzzyquery[dffuzzyquery['CT_Current_DT'].notna()]
```

```
C:\Users\EIJEL\AppData\Local\Temp\ipykernel_27372\579294774.py:1: DtypeWarning: Columns (35,81,137,139) have mixed types. Specify dtype
option on import or set low_memory=False.
  dffuzzyquery = pd.read_csv(r'C:\Users\EIJEL\OneDrive - Bayer\Personal Data\Erik\2024\pkl_new\cotton-
dormancy\data\truck_to_quality\fuzzy_lineage_query.csv')
```

In [9]:

```
sorted(dffuzzyquery.columns)
```

Out [9]:

```
['ABA_Level',
 'Actual_Seed_Per_LB',
 'Appearance_Rating',
 'BLACK_ORD_TYP_CD',
 'BLACK_PLNT_CD',
 'BLACK_PRCS_ORD_NBR',
 'CT_Current',
 'CT_Current_DT',
 'CT_Date_Abn',
 'CT_Date_Dead',
 'CT_Date_FH',
 'CT_Date_Resample',
 'CT_Date_Resample_Abn',
 'CT_Date_Resample_DT',
 'CT_Date_Resample_Dead',
 'CT_Date_Resample_FH',
 'CT_Date_Resample_SS',
 'CT_Date_SS',
 'CT_Initial',
 'CT_Initial_Abn',
 'CT_Initial_DT',
 'CT_Initial_Dead',
 'CT_Initial_FH',
 'CT_Initial_Resample',
 'CT_Initial_Resample_Dead',
 'CT_Initial_SS',
 'CT_Test',
 'CT_Test_DT',
 'CT_Testing_Complete',
 'Cleanout_PCT',
 'Clothianidin',
 'Cry1Ac_Bollgard_Strip_Test',
 'Cry2Ab_Bollgard_Strip_Test',
 'Cut_Test_Discolored',
 'Cut_Test_Immature',
 'DAY4_CNT_IN_WG1',
 'DAY4_CNT_IN_WG2',
 'DAY7_CNT_IN_WG3',
 'DAY7_CNT_IN_WG3_DT',
 'Delintability',
 'Dicamba_Lateral_Strip_PCT',
 'Dust_Off',
 'External_2_4_D_Strip_Test',
 'External_Cry1Ac_Strip_Test',
 'External_Cry1F_Strip_Test',
 'External_Production_IL',
 'External_RR_Strip_Test',
 'External_VIP3A_Strip_Test',
 'FFA',
 'FIELD',
```

```
'FUZZY_BATCH_NBR',
'FUZZY_BRND_DESC',
'FUZZY_CROP_CD',
'FUZZY_GI_QTY',
'FUZZY_GI_QTY_SSU',
'FUZZY_GR_QTY',
'FUZZY_GR_QTY_SSU',
'FUZZY_INSPCT_LOT_NBR',
'FUZZY_MTRL_DESC',
'FUZZY_MTRL_GRP_NM',
'FUZZY_MTRL_NBR',
'FUZZY_MTRL_TYP_CD',
'FUZZY_PKG_TYP_DESC',
'FUZZY_PLNT_CD',
'FUZZY_PRCS_ORD_NBR',
'FUZZY_TRAIT',
'FUZZY_VRTY_DESC',
'Fluxapyroxad',
'GRWNG_CNTRCT',
'GRWR_PYMNT',
'Heavy_Mechanical_Damage',
'Imidacloprid',
'Inert_Matter',
'LOAD_BATCH_NBR',
'LOAD_GI_QTY',
'LOAD_GI_QTY_SSU',
'LOAD_GR_QTY',
'LOAD_GR_QTY_SSU',
'LOAD_MTRL_NBR',
'LOAD_PLNT_CD',
'LOT_CREATED_DT',
'Liberty_Lateral_Strip_PCT',
'Light_Mechanical_Damage',
'Lint_Loss_PCT',
'Lygus_PCT',
'Mechanical_Damage',
'Medium_Mechanical_Damage',
'Metalaxyl',
'Moisture',
'Myclobutanil',
'ORD_TYP_CD',
'ORIGIN_REGION',
'PA_ID',
'PURCH_ORD_NBR',
'Pure_Seed',
'Pyraclostrobin',
'QM_UD_CD',
'RR_Lateral_Strip_PCT',
'SEASON_YR',
'Seed_Temperature',
'State_Lab_Cool',
'State_Lab_Final_Germ',
'State_Lab_Final_Germ_DT',
'Tetrazolium_High_Vigor',
'Tetrazolium_Predicted_Germ',
'USBL_UNITS_SSU',
'USG_DECSN_INSPCT_CD',
'VIP3A_Bollgard_Strip_Test',
'Vigor_ABN_121',
'Vigor_FH_123',
'Vigor_SS_122',
'WG3_Date_ABN',
'WG3_Date_ABN_DT',
'WG3_Date_DEAD',
'WG3_Date_DEAD_DT',
'WG3_Date_FH',
'WG3_Date_FH_DT',
'WG_Current',
'WG_Current_DT',
'WG_Date_7Day_Cnt',
'WG_Date_7Day_Cnt_DT',
'WG_Date_Abn',
'WG_Date_Dead',
'WG_Date_FH',
'WG_Date_Resample_7Day_Cnt',
'WG_Date_Resample_7Day_Cnt_DT',
'WG_Date_Resample_Abn',
'WG_Date_Resample_Dead',
'WG_Date_Resample_FH',
'WG_Date_Resample_SS',
'WG_Date_SS',
'WG_Initial_7Day_Cnt',
'WG_Initial_7Day_Cnt_DT',
'WG_Initial_Abn',
'WG_Initial_Dead',
'WG_Initial_FH',
'WG_Initial_Resample_7Day_Cnt',
'WG_Initial_Resample_7Day_Cnt_DT',
'WG_Initial_Resample_Abn',
'WG_Initial_Resample_Dead',
'WG_Initial_Resample_FH',
'WG_Initial_Resample_SS',
'WG_Initial_SS',
'WG_Testing_Complete',
'Xdamage',
'Xmaturity',
'Xtexture',
'agreement_typ_cd',
'pH',
'row_number',
'spcl_trtmnt_cd']
```

In [26]:

```
dffuzzyquery.head()
```

Out [26]:

	SEASON_YR	GRWNG_CNTRCT	PURCH_ORD_NBR	FIELD	GRWR_PYMNT	FUZZY_PRCS_ORD_NBR	ORD_TYP_CD	agreement_typ_cd	LOAD_MTRL_N
3	2020	4600000072	613322461.0	1646-O	627875.91	2502130823	ZG20	ZTT2	12269510

	SEASON_YR	GRWNG_CNTRCT	PURCH_ORD_NBR	FIELD	GRWR_PYMNT	FUZZY_PRCS_ORD_NBR	ORD_TYP_CD	agreement_typ_cd	LOAD_MTRL_N
7	2020	4600000060	613300400.0	1646-O	399918.64	2502133098	ZG20	ZTT2	12269510
8	2020	4600000060	613300400.0	1820-O	399918.64	2502133099	ZG20	ZTT2	12528885
12	2020	4608651771	613315791.0	1522-F	934190.50	2502138421	ZG21	ZTA2	12206829
14	2020	4600000010	613322447.0	1822-O	275838.20	2502130272	ZG20	ZTT2	12528569

In [5]:
```
df_final = dffuzzyquery.merge(cotton_with_truck, left_on='FUZZY_INSPCT_LOT_NBR', right_on='inspct_lot_nbr')
```

In [6]:
```
df_final.shape
```

Out [6]:
```
(922, 251)
```

In [7]:
```
df_final.SEASON_YR.value_counts()
```

Out [7]:
```
SEASON_YR
2023    272
2020    200
2024    190
2021    143
2022    117
Name: count, dtype: int64
```

In [8]:
```
df_single_batch_latest = df_final.copy()
```

In [14]:
```
########################################### ignore above code ###########################################
```

In [9]:
```
df_single_batch_latest.harvest_date = pd.to_datetime(df_single_batch_latest.harvest_date)
df_single_batch_latest.CT_Current_DT = pd.to_datetime(df_single_batch_latest.CT_Current_DT)
df_single_batch_latest.WG_Current_DT = pd.to_datetime(df_single_batch_latest.WG_Current_DT)
```

In [10]:
```
df_single_batch_latest['harvest_to_CT_dt'] = df_single_batch_latest.CT_Current_DT - df_single_batch_latest.harv
```

In [11]:
```
df_single_batch_latest['harvest_to_WG_dt'] = df_single_batch_latest.WG_Current_DT - df_single_batch_latest.harv
```

In [14]:
```
def convert_days_to_int(series):
    """Convert 'n days' strings to integers (e.g., '5 days' -> 5).
    Handles pandas.NA for missing integer values.
    """
    if not isinstance(series, pd.Series):
        return series # Return as is if not a Series

    # Apply only if the series contains strings
    if series.dtype == 'object' or pd.api.types.is_string_dtype(series):
        return pd.to_numeric(series.str.replace(' days', '', regex=False), errors='coerce').astype('Int64')
    if pd.api.types.is_timedelta64_dtype(series.dtype):
        return series.map(lambda x: x.days)
    return series # Return original series if not string type

df_single_batch_latest.defol_to_harvest = convert_days_to_int(df_single_batch_latest.defol_to_harvest)
df_single_batch_latest.harvest_to_CT_dt = convert_days_to_int(df_single_batch_latest.harvest_to_CT_dt)
df_single_batch_latest.harvest_to_WG_dt = convert_days_to_int(df_single_batch_latest.harvest_to_WG_dt)
```

In [15]:
```
df_single_batch_latest.head()
```

Out [15]:

	SEASON_YR	GRWNG_CNTRCT	PURCH_ORD_NBR	FIELD	GRWR_PYMNT	FUZZY_PRCS_ORD_NBR_x	ORD_TYP_CD	agreement_typ_cd	LOAD_MTRL_
0	2020	4600000106	613300418.0	1822-O	70803.28	2502133229	ZG20	ZTT2	12528569
1	2021	4600006241	614130422.0	2141-O	581004.20	2502245532	ZG20	ZTT2	87884902
2	2021	4600006241	614130422.0	2141-O	581004.20	2502245532	ZG20	ZTT2	87884902

	SEASON_YR	GRWNG_CNTRCT	PURCH_ORD_NBR	FIELD	GRWR_PYMNT	FUZZY_PRCS_ORD_NBR_x	ORD_TYP_CD	agreement_typ_cd	LOAD_MTRL_
3	2021	4600006294	614127280.0	2038-O	1167922.60	2502245956	ZG20	ZTT2	12747566
4	2022	4600012576	615014520.0	R649-E	81585.95	2502363371	ZG21	ZTA3	88809017

In [ ]:
```
df_single_batch_prev_with_20232024 = pd.read_csv(r'C:\Users\EIJEL\OneDrive - Bayer\Personal Data\Erik\2024\pkl_
```

In [ ]:
```
df_single_batch_latest.to_csv('field_to_facility.csv', index=False)
```

In [20]:
```
df_single_batch_latest.SEASON_YR.value_counts()
```

Out [20]:
```
SEASON_YR
2023    272
2020    200
2024    190
2021    143
2022    117
Name: count, dtype: int64
```

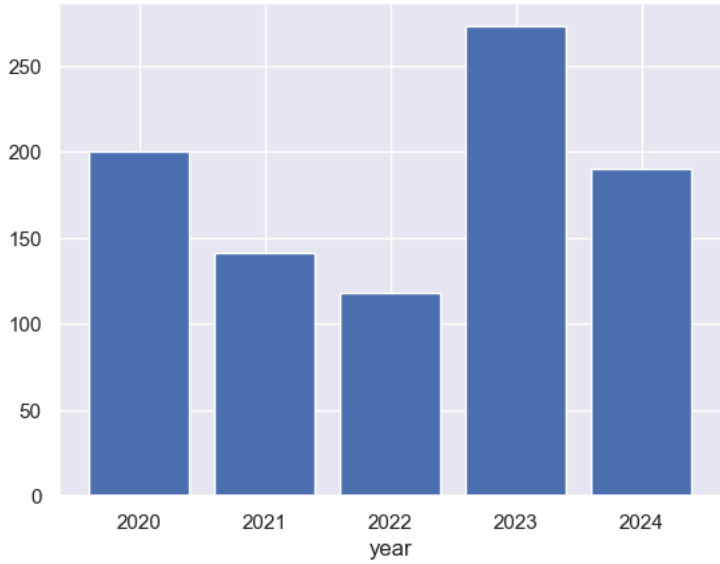
In [16]:
```
# df_single_batch = df_single_batch_prev_with_20232024
df_single_batch = df_single_batch_latest.copy()
```

In [22]:
```
df_single_batch.head()
```

Out [22]:

	SEASON_YR	GRWNG_CNTRCT	PURCH_ORD_NBR	FIELD	GRWR_PYMNT	FUZZY_PRCS_ORD_NBR_x	ORD_TYP_CD	agreement_typ_cd	LOAD_MTRL_
0	2020	4600000106	613300418.0	1822-O	70803.28	2502133229	ZG20	ZTT2	12528569
1	2021	4600006241	614130422.0	2141-O	581004.20	2502245532	ZG20	ZTT2	87884902
2	2021	4600006241	614130422.0	2141-O	581004.20	2502245532	ZG20	ZTT2	87884902
3	2021	4600006294	614127280.0	2038-O	1167922.60	2502245956	ZG20	ZTT2	12747566
4	2022	4600012576	615014520.0	R649-E	81585.95	2502363371	ZG21	ZTA3	88809017

In [17]:
```
# df_single_batch.date
import seaborn as sns
sns.set_theme()
df_single_batch['date'] = pd.to_datetime(df_single_batch['date'])

# Extract the year and get the value counts
year_counts = df_single_batch['date'].dt.year.value_counts()
df_single_batch['year'] = df_single_batch['date'].dt.year
# print(year_counts)
plt.bar(year_counts.index,year_counts)
plt.xlabel('year')
plt.xticks([2020, 2021, 2022, 2023, 2024])
plt.show()
# plt.yscale('log')
# plt.savefig('year_dist.png', dpi = 600)
```

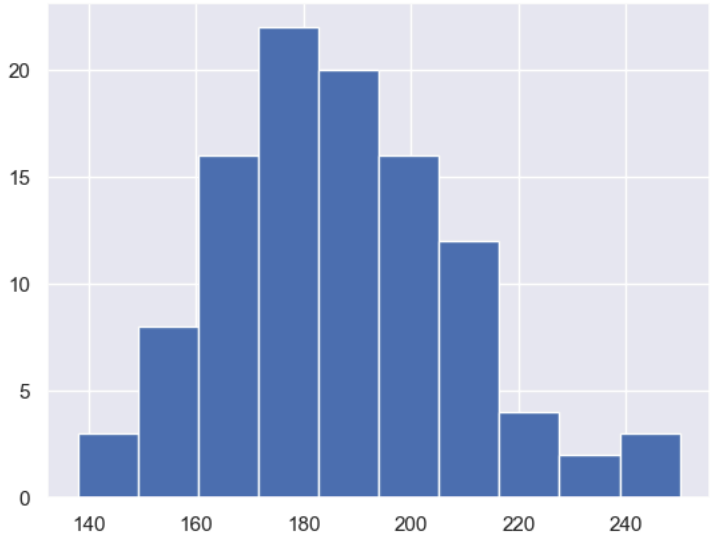


In [28]:
```
# lets use all cols
cols_to_train = [
 'irrigation_type',
 'ans_variety',
 'cumulated_dd60',
 'cumulated_soil_moisture',
 'dd60',
 'defol_to_harvest',
 'defol_to_harvest_average_dd60',
 'defol_to_harvest_average_heat',
 'defol_to_harvest_cum_dd60',
 'defol_to_harvest_cum_heat',
    # 'harvest_to_CT_dt',
    # 'harvest_to_WG_dt',
    "maczone",
  'pp_day10_avg_dd60',
 'pp_day10_avg_soilmoisture',
 'pp_day10_cum_dd60',
 'pp_day10_cum_soilmoisture',
 'pp_day5_avg_dd60',
 'pp_day5_avg_soilmoisture',
 'pp_day5_cum_dd60',
 'pp_day5_cum_soilmoisture',
    # 'season_length',
    # 'catch_rate',
     'CT_Current',
 # 'WG_Current',
    'year'
]

dftotrain = df_single_batch[cols_to_train]
# dftotrain = dftotrain[(dftotrain.CT_Initial.notna()) & \
# (dftotrain.WG_Initial_7Day_Cnt.notna()) & \
# (dftotrain.irrigation_type.notna()) & \
# (dftotrain.ans_variety.notna()) & \
# (dftotrain.defol_to_harvest.notna())
# ]

dftotrain = dftotrain.dropna()
```

In [31]:
```
# dftotrain.catch_rate.value_counts()
```

In [24]:
```
dftotrain[dftotrain.year == 2022].season_length.hist()
```

Out [24]: <Axes: >



In [41]:
```
cols_to_train = [
 'irrigation_type',
 'ans_variety',
 'cumulated_dd60',
 'cumulated_soil_moisture',
 'dd60',
 'defol_to_harvest',
 'defol_to_harvest_average_dd60',
 'defol_to_harvest_average_heat',
 'defol_to_harvest_cum_dd60',
 'defol_to_harvest_cum_heat',
    'harvest_to_CT_dt',
    'harvest_to_WG_dt',
    "maczone",
  # 'pp_day10_avg_dd60',
 # 'pp_day10_avg_soilmoisture',
 # 'pp_day10_cum_dd60',
 # 'pp_day10_cum_soilmoisture',
 # 'pp_day5_avg_dd60',
 'pp_day5_avg_soilmoisture',
 # 'pp_day5_cum_dd60',
 # 'pp_day5_cum_soilmoisture',
    # 'season_length',
    # 'catch_rate',
     'CT_Current',
 'WG_Current',
    'year'
]

dftotrain = df_single_batch[cols_to_train]
# dftotrain = dftotrain[(dftotrain.CT_Initial.notna()) & \
# (dftotrain.WG_Initial_7Day_Cnt.notna()) & \
# (dftotrain.irrigation_type.notna()) & \
# (dftotrain.ans_variety.notna()) & \
# (dftotrain.defol_to_harvest.notna())
# ]

dftotrain = dftotrain.dropna()
```

In [42]:
```
dftotrain.head()
```

Out [42]:

	irrigation_type	ans_variety	cumulated_dd60	cumulated_soil_moisture	dd60	defol_to_harvest	defol_to_harvest_average_dd60	defol_to_h
0	CenterPivotNBC	DP1822XF	2675.05	27.735	0.60	21	2.361364	12.060182
1	CenterPivotLEPA	DP2141NRB3XF	2445.25	17.015	4.30	26	3.200000	12.167926
2	CenterPivotLEPA	DP2141NRB3XF	2477.30	15.535	6.25	18	4.105263	12.088632
3	CenterPivotNBC	DP2038B3XF	3128.50	34.485	7.10	28	5.931034	11.818241
4	Flood	21R649NRB3XF_NPE	4862.85	20.040	0.00	52	1.404717	10.455792

In [ ]:
```
dftotrain.head()
```

In [26]:
```
dftotrain.isna().sum()
```

Out [26]:
```
irrigation_type                 0
ans_variety                     0
cumulated_dd60                  0
cumulated_soil_moisture         0
dd60                            0
defol_to_harvest                0
defol_to_harvest_average_dd60   0
defol_to_harvest_average_heat   0
defol_to_harvest_cum_dd60       0
defol_to_harvest_cum_heat       0
harvest_to_CT_dt                0
harvest_to_WG_dt                0
maczone                         0
pp_day5_avg_soilmoisture        0
CT_Current                      0
WG_Current                      0
year                            0
dtype: int64
```

In [18]:
```
# classification
cold_class = {'CT_Current': dftotrain.CT_Current >= 60.0, 'WG_Current': dftotrain.WG_Current >= 90.0}
```

In [ ]:
```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, GridSearchCV, cross_val_score
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestRegressor
from lightgbm import LGBMRegressor
from xgboost import XGBRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import warnings
warnings.filterwarnings('ignore')
warnings.filterwarnings("ignore", category=UserWarning)

# Define base features (excluding both target columns and target-specific date features)

base_features = [col for col in cols_to_train if col not in ['CT_Current', 'WG_Current', 'harvest_to_CT_dt', '

# Define target-specific feature sets
feature_sets = {
    'CT_Current': base_features + ['harvest_to_CT_dt'],
    'WG_Current': base_features + ['harvest_to_WG_dt']
}

# Define binary targets
targets = ['CT_Current']

# Preprocess features for each target
X_dict = {}
label_encoders = {}
for target_name in targets:
    features = feature_sets[target_name]
    X = dftotrain[features].copy()

    # Identify categorical and numerical columns
    categorical_cols = X.select_dtypes(include=['object']).columns
    numerical_cols = X.select_dtypes(include=['float64', 'int64']).columns

    # Encode categorical variables (including harvest_to_*_dt if categorical)
    for col in categorical_cols:
        le = LabelEncoder()
        X[col] = le.fit_transform(X[col].astype(str))
        label_encoders[f"{target_name}_{col}"] = le

    # Handle missing values (if any)
    X[numerical_cols] = X[numerical_cols].fillna(X[numerical_cols].mean())
    X[categorical_cols] = X[categorical_cols].fillna('missing')

    X_dict[target_name] = X

# Define parameter grids for grid search
param_grids = {
    'Random Forest': {
        "n_estimators": [25, 40, 50, 60, 80],
        "max_features": ["sqrt"],
        "max_depth": [3, 5, 7, 9],
```

```python
        "min_samples_split": [2, 3, 5],
        "min_samples_leaf": [1, 2, 4],
    },
    'LightGBM': {
        'n_estimators': [25, 50],
        'max_depth': [1, 4, -1],
        'learning_rate': [0.01, 0.1],
        'verbose': [-1]
    },
    'XGBoost': {
        'n_estimators': [25, 50],
        'max_depth': [3, 4, 5],
        'learning_rate': [0.01, 0.1]
    }
}

# Initialize models
model_classes = {
    'Random Forest': RandomForestRegressor(random_state=42),
    'LightGBM': LGBMRegressor(random_state=42, verbose=-1),
    'XGBoost': XGBRegressor(random_state=42)
}

# Function to perform grid search and evaluate model
def train_and_evaluate(X, y, model_name, model, param_grid, target_name):
    # Split data
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.4, random_state=42)

    # Grid search
    grid_search = GridSearchCV(
        estimator=model,
        param_grid=param_grid,
        cv=5,
        scoring='r2',
        n_jobs=-1
    )
    grid_search.fit(X_train, y_train)

    # Best model
    best_model = grid_search.best_estimator_
    print(f"\n{model_name} Best Parameters for {target_name}: {grid_search.best_params_}")
    print(f"Best CV R2 Score: {grid_search.best_score_:.4f}")

    # Predict on train and test sets
    y_train_pred = best_model.predict(X_train)
    y_test_pred = best_model.predict(X_test)

    # Calculate metrics
    train_mse = mean_squared_error(y_train, y_train_pred)
    train_mae = mean_absolute_error(y_train, y_train_pred)
    train_r2 = r2_score(y_train, y_train_pred)
    test_mse = mean_squared_error(y_test, y_test_pred)
    test_mae = mean_absolute_error(y_test, y_test_pred)
    test_r2 = r2_score(y_test, y_test_pred)

    # Cross-validation on best model
    cv_scores = cross_val_score(best_model, X_train, y_train, cv=5, scoring='r2')

    # Store results
    results = {
        'Train MSE': train_mse,
        'Train MAE': train_mae,
        'Train R2': train_r2,
        'Test MSE': test_mse,
        'Test MAE': test_mae,
        'Test R2': test_r2,
        'CV R2 Mean': cv_scores.mean(),
        'CV R2 Std': cv_scores.std(),
        'Best Parameters': grid_search.best_params_
    }

    # Feature importance
    if model_name == 'Random Forest':
```

```
        feature_importance = pd.DataFrame({
            'Feature': X.columns,
            'Importance': best_model.feature_importances_
        }).sort_values(by='Importance', ascending=False)
        print(f"\nFeature Importance for {model_name} ({target_name}):")
        print(feature_importance)
    elif model_name == 'XGBoost':
        importance_dict = best_model.get_booster().get_score(importance_type='gain')
        # Initialize importance list with zeros for all features
        importance_values = [importance_dict.get(col, 0.0) for col in X.columns]
        feature_importance = pd.DataFrame({
            'Feature': X.columns,
            'Importance': importance_values
        }).sort_values(by='Importance', ascending=False)
        print(f"\nFeature Importance for {model_name} ({target_name}):")
        print(feature_importance)

    return results

# Train and evaluate models for each target
all_results = {}
for target in targets:
    print(f"\n=== Training Models for Target: {target} ===")
    y = dftotrain[target]
    X = X_dict[target_name]
    all_results[target] = {}

    for model_name, model in model_classes.items():
        results = train_and_evaluate(X, y, model_name, model, param_grids[model_name], target)
        all_results[target][model_name] = results

        print(f"\n{model_name} Results for {target}:")
        print(f"Train MSE: {results['Train MSE']:.4f}")
        print(f"Train MAE: {results['Train MAE']:.4f}")
        print(f"Test MSE: {results['Test MSE']:.4f}")
        print(f"Test MAE: {results['Test MAE']:.4f}")
        print(f"Test R2: {results['Test R2']:.4f}")
        print(f"Cross-Validation R2: {results['CV R2 Mean']:.4f} (+/- {results['CV R2 Std'] * 2:.4f})")

# Summarize results
for target in targets:
    print(f"\nSummary of Results for {target}:")
    results_df = pd.DataFrame(all_results[target]).T
    print(results_df[['Train MSE', 'Train MAE', 'Train R2', 'Test MSE', 'Test MAE', 'Test R2', 'CV R2 Mean', '(
```

In [43]:

```
groups = dftotrain.year
# dftotrain = dftotrain[dftotrain.year != 2023]
```

In [44]:

```
dftotrain.dtypes
```

Out [44]:

```
irrigation_type                  object
ans_variety                      object
cumulated_dd60                  float64
cumulated_soil_moisture         float64
dd60                            float64
defol_to_harvest                  Int64
defol_to_harvest_average_dd60   float64
defol_to_harvest_average_heat   float64
defol_to_harvest_cum_dd60       float64
defol_to_harvest_cum_heat       float64
harvest_to_CT_dt                  int64
harvest_to_WG_dt                float64
maczone                         float64
pp_day5_avg_soilmoisture        float64
CT_Current                      float64
WG_Current                      float64
year                              int32
dtype: object
```

In [45]:

```
import pandas as pd
import joblib
import numpy as np
from sklearn.model_selection import train_test_split, GridSearchCV, cross_val_score, GroupKFold
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestRegressor
from lightgbm import LGBMRegressor
from xgboost import XGBRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import warnings
```

```python
import matplotlib.pyplot as plt
import seaborn as sns

warnings.filterwarnings('ignore')
warnings.filterwarnings("ignore", category=UserWarning)

# Define base features (excluding both target columns and target-specific date features)
base_features = [col for col in cols_to_train if col not in ['CT_Current', 'WG_Current', 'harvest_to_CT_dt', 'h

# Define target-specific feature sets
feature_sets = {
    'CT_Current': base_features + ['harvest_to_CT_dt'],
    'WG_Current': base_features + ['harvest_to_WG_dt']
}

# Define binary targets
targets = ['CT_Current']

# Preprocess features for each target
X_dict = {}
label_encoders = {}
for target_name in targets:
    features = feature_sets[target_name]
    X = dftotrain[features].copy()

    # Identify categorical and numerical columns
    categorical_cols = X.select_dtypes(include=['object']).columns
    numerical_cols = X.select_dtypes(include=['float64', 'int64']).columns

    # Encode categorical variables (including harvest_to_*_dt if categorical)
    for col in categorical_cols:
        le = LabelEncoder()
        X[col] = le.fit_transform(X[col].astype(str))
        label_encoders[f"{target_name}_{col}"] = le

    # Handle missing values (if any)
    X[numerical_cols] = X[numerical_cols].fillna(X[numerical_cols].mean())
    X[categorical_cols] = X[categorical_cols].fillna('missing')

    X_dict[target_name] = X

# Save the label encoders dictionary to a pickle file
joblib.dump(label_encoders, 'label_encoders.pkl')
print("Label encoders saved to 'label_encoders.pkl'")

# To retrieve later (example usage):
# loaded_label_encoders = joblib.load('label_encoders.pkl')
# For a specific encoder, e.g., le = loaded_label_encoders['CT_Current_some_column']


# Define parameter grids for grid search
param_grids = {
    'Random Forest': {
        "n_estimators": [25, 40, 50, 60, 80],
        "max_features": ["sqrt"],
        "max_depth": [3, 5, 7, 9],
        "min_samples_split": [2, 3, 5],
        "min_samples_leaf": [1, 2, 4],
    },
    'LightGBM': {
        'n_estimators': [25, 50],
        'max_depth': [1, 4, -1],
        'learning_rate': [0.01, 0.1],
        'verbose': [-1]
    },
    'XGBoost': {
        'n_estimators': [25, 50],
        'max_depth': [3, 4, 5],
        'learning_rate': [0.01, 0.1]
    }
}

# Initialize models
```

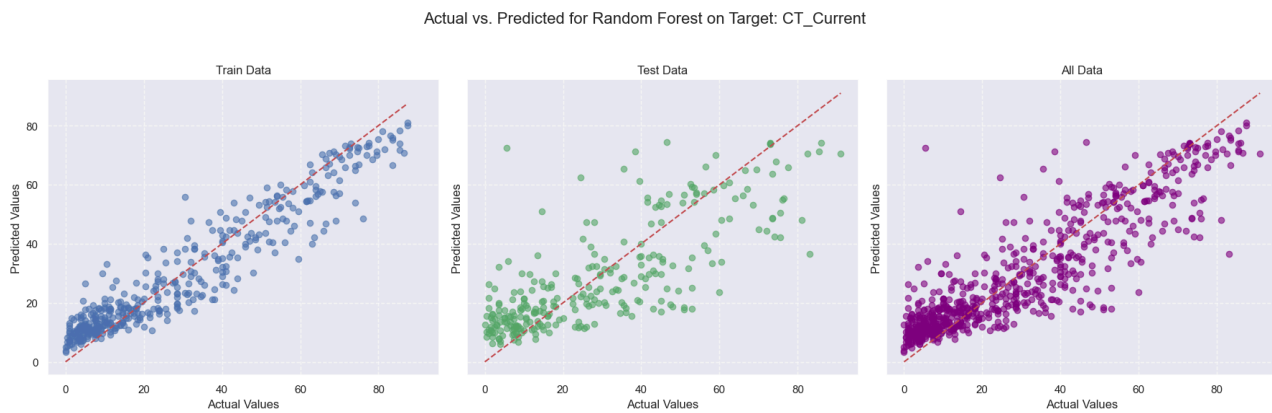
```python
model_classes = {
    'Random Forest': RandomForestRegressor(random_state=42),
    'LightGBM': LGBMRegressor(random_state=42, verbose=-1),
    'XGBoost': XGBRegressor(random_state=42)
}

# Function to perform grid search and evaluate model
def train_and_evaluate(X, y, model_name, model, param_grid, target_name):
    groups = dftotrain.year
    gkf = GroupKFold(n_splits=len(groups.unique()))
    # Split data
    X_train, X_test, y_train, y_test, groups_train, groups_test = train_test_split(
        X, y, groups, test_size=0.4, random_state=42
    )
    # X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.4, random_state=42)

    # Grid search
    grid_search = GridSearchCV(
        estimator=model,
        param_grid=param_grid,
        cv=gkf,
        scoring='r2',
        n_jobs=-1
    )
    grid_search.fit(X_train, y_train, groups=groups_train)

    # Best model
    best_model = grid_search.best_estimator_
    print(f"\n{model_name} Best Parameters for {target_name}: {grid_search.best_params_}")
    print(f"Best CV R2 Score: {grid_search.best_score_:.4f}")
    joblib.dump(best_model, f'model_{target_name}_{model_name.lower().replace(" ", "_")}.pkl')

    # Predict on train and test sets
    y_train_pred = best_model.predict(X_train)
    y_test_pred = best_model.predict(X_test)
    y_full_pred = best_model.predict(X)

    # Calculate metrics
    train_mse = mean_squared_error(y_train, y_train_pred)
    train_mae = mean_absolute_error(y_train, y_train_pred)
    train_r2 = r2_score(y_train, y_train_pred)
    test_mse = mean_squared_error(y_test, y_test_pred)
    test_mae = mean_absolute_error(y_test, y_test_pred)
    test_r2 = r2_score(y_test, y_test_pred)

    # Cross-validation on best model
    cv_scores = cross_val_score(best_model, X_train, y_train, cv=5, scoring='r2')

    # Store results
    results = {
        'Train MSE': train_mse,
        'Train MAE': train_mae,
        'Train R2': train_r2,
        'Test MSE': test_mse,
        'Test MAE': test_mae,
        'Test R2': test_r2,
        'CV R2 Mean': cv_scores.mean(),
        'CV R2 Std': cv_scores.std(),
        'Best Parameters': grid_search.best_params_
    }

    # Store predictions for plotting
    predictions = {
        'y_train': y_train,
        'y_train_pred': y_train_pred,
        'y_test': y_test,
        'y_test_pred': y_test_pred,
        'y_full': y,
        'y_full_pred': y_full_pred
    }

    # Feature importance
    if model_name == 'Random Forest':
```

```python
        feature_importance = pd.DataFrame({
            'Feature': X.columns,
            'Importance': best_model.feature_importances_
        }).sort_values(by='Importance', ascending=False)
        print(f"\nFeature Importance for {model_name} ({target_name}):")
        print(feature_importance)
    elif model_name == 'XGBoost':
        importance_dict = best_model.get_booster().get_score(importance_type='gain')
        # Initialize importance list with zeros for all features
        importance_values = [importance_dict.get(col, 0.0) for col in X.columns]
        feature_importance = pd.DataFrame({
            'Feature': X.columns,
            'Importance': importance_values
        }).sort_values(by='Importance', ascending=False)
        print(f"\nFeature Importance for {model_name} ({target_name}):")
        print(feature_importance)

    return results, predictions, best_model

# Function to plot actual vs. predicted values
def plot_predictions(model_name, target_name, predictions):
    y_train, y_train_pred = predictions['y_train'], predictions['y_train_pred']
    y_test, y_test_pred = predictions['y_test'], predictions['y_test_pred']
    y_full, y_full_pred = predictions['y_full'], predictions['y_full_pred']

    fig, axes = plt.subplots(1, 3, figsize=(18, 6), sharex=True, sharey=True)
    fig.suptitle(f'Actual vs. Predicted for {model_name} on Target: {target_name}', fontsize=16)

    # Plot Train Data
    axes[0].scatter(y_train, y_train_pred, alpha=0.6, color='b')
    axes[0].plot([y_train.min(), y_train.max()], [y_train.min(), y_train.max()], 'r--')
    axes[0].set_title('Train Data')
    axes[0].set_xlabel('Actual Values')
    axes[0].set_ylabel('Predicted Values')
    axes[0].grid(True, linestyle='--', alpha=0.6)

    # Plot Test Data
    axes[1].scatter(y_test, y_test_pred, alpha=0.6, color='g')
    axes[1].plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--')
    axes[1].set_title('Test Data')
    axes[1].set_xlabel('Actual Values')
    axes[1].set_ylabel('Predicted Values')
    axes[1].grid(True, linestyle='--', alpha=0.6)

    # Plot All Data
    axes[2].scatter(y_full, y_full_pred, alpha=0.6, color='purple')
    axes[2].plot([y_full.min(), y_full.max()], [y_full.min(), y_full.max()], 'r--')
    axes[2].set_title('All Data')
    axes[2].set_xlabel('Actual Values')
    axes[2].set_ylabel('Predicted Values')
    axes[2].grid(True, linestyle='--', alpha=0.6)

    plt.tight_layout(rect=[0, 0.03, 1, 0.95])
    plt.savefig(f'{model_name}_{target_name}_actual_vs_predicted.png')
    plt.show()

# Train and evaluate models for each target
all_results = {}
all_predictions = {}

for target in targets:
    print(f"\n=== Training Models for Target: {target} ===")
    y = dftotrain[target]
    X = X_dict[target]
    all_results[target] = {}
    all_predictions[target] = {}
    model_best = {}

    for model_name, model in model_classes.items():
        results, predictions, model_ = train_and_evaluate(X, y, model_name, model, param_grids[model_name], ta
        model_best[model_name] = model_
        all_results[target][model_name] = results
        all_predictions[target][model_name] = predictions
```

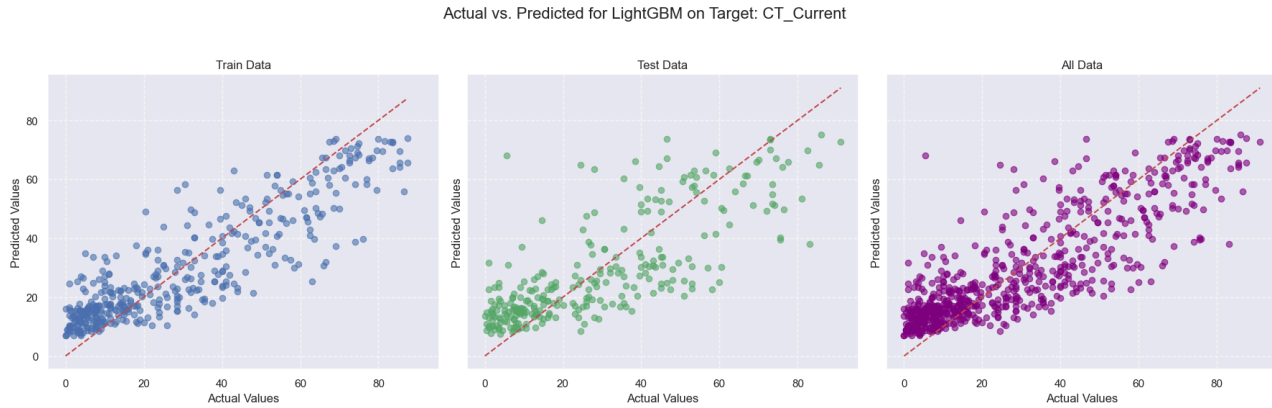
```
    print(f"\n{model_name} Results for {target}:")
    print(f"Train MSE: {results['Train MSE']:.4f}")
    print(f"Train MAE: {results['Train MAE']:.4f}")
    print(f"Test MSE: {results['Test MSE']:.4f}")
    print(f"Test MAE: {results['Test MAE']:.4f}")
    print(f"Test R2: {results['Test R2']:.4f}")
    print(f"Cross-Validation R2: {results['CV R2 Mean']:.4f} (+/- {results['CV R2 Std'] * 2:.4f})")

    # Plot predictions for the current model
    plot_predictions(model_name, target, predictions)

# Get feature importance for Random Forest
rf_model = model_best['Random Forest']
rf_importance_df = pd.DataFrame({
    'Feature': X.columns,
    'Importance': rf_model.feature_importances_
})

# Get feature importance for XGBoost
xgb_model = model_best['XGBoost']
importance_dict = xgb_model.get_booster().get_score(importance_type='gain')
importance_values = [importance_dict.get(col, 0.0) for col in X.columns]
xgb_importance_df = pd.DataFrame({
    'Feature': X.columns,
    'Importance': importance_values
})

# Normalize importance to sum to 100% for comparison
rf_importance_df['Importance'] = (rf_importance_df['Importance'] / rf_importance_df['Importance'].sum()) *
xgb_importance_df['Importance'] = (xgb_importance_df['Importance'] / xgb_importance_df['Importance'].sum()

# Add a 'Model' column to each DataFrame
rf_importance_df['Model'] = 'Random Forest'
xgb_importance_df['Model'] = 'XGBoost'

# Combine the two dataframes
combined_importance_df = pd.concat([rf_importance_df, xgb_importance_df])

# Sort the data by Random Forest importance for consistent ordering
combined_importance_df['Sort_Order'] = combined_importance_df['Feature'].map(rf_importance_df.set_index('F
combined_importance_df.sort_values(by=['Sort_Order', 'Model'], ascending=[False, True], inplace=True)

# Create the bar chart
plt.figure(figsize=(12, 8))
sns.barplot(
    x='Importance',
    y='Feature',
    hue='Model',
    data=combined_importance_df
)

plt.title(f'Comparison of Feature Importance for Random Forest and XGBoost on {target}', fontsize=16)
plt.xlabel('Relative Importance (%)', fontsize=12)
plt.ylabel('Features', fontsize=12)
plt.legend(title='Model')
plt.tight_layout()

# Optional: Save the figure
plt.savefig(f'{target}_feature_importance_comparison.png')
plt.show()

plt.figure(figsize=(8, 6))
sns.barplot(
    x='Importance',
    y='Feature',
    data=xgb_importance_df.sort_values(by='Importance', ascending=False).iloc[:10,:]
)
plt.title(f'Comparison of Feature Importance for XGBoost on {target}', fontsize=12)
plt.xlabel('Relative Importance (%)', fontsize=12)
plt.ylabel('Features', fontsize=12)
plt.tight_layout()
```

```python
    # Optional: Save the figure
    plt.savefig(f'{target}_feature_importance_xgboost.png')
    plt.show()

# Summarize results
for target in targets:
    print(f"\nSummary of Results for {target}:")
    results_df = pd.DataFrame(all_results[target]).T
    print(results_df[['Train MSE', 'Train MAE', 'Train R2', 'Test MSE', 'Test MAE', 'Test R2', 'CV R2 Mean', '(
```

```
Label encoders saved to 'label_encoders.pkl'

=== Training Models for Target: CT_Current ===

Random Forest Best Parameters for CT_Current: {'max_depth': 9, 'max_features': 'sqrt', 'min_samples_leaf': 1, 'min_samples_split': 3, 'n_estimators':
Best CV R2 Score: 0.3625

Feature Importance for Random Forest (CT_Current):
                        Feature  Importance
8       defol_to_harvest_cum_dd60    0.178374
6   defol_to_harvest_average_dd60    0.163824
2                  cumulated_dd60    0.161559
10                        maczone    0.081399
12               harvest_to_CT_dt    0.068438
1                     ans_variety    0.066219
3         cumulated_soil_moisture    0.060065
7   defol_to_harvest_average_heat    0.053504
11       pp_day5_avg_soilmoisture    0.047483
4                            dd60    0.036008
0                 irrigation_type    0.029997
5                defol_to_harvest    0.026952
9       defol_to_harvest_cum_heat    0.026178

Random Forest Results for CT_Current:
Train MSE: 63.7628
Train MAE: 6.3841
Test MSE: 204.8765
Test MAE: 10.8177
Test R2: 0.6043
Cross-Validation R2: 0.6193 (+/- 0.0982)
```

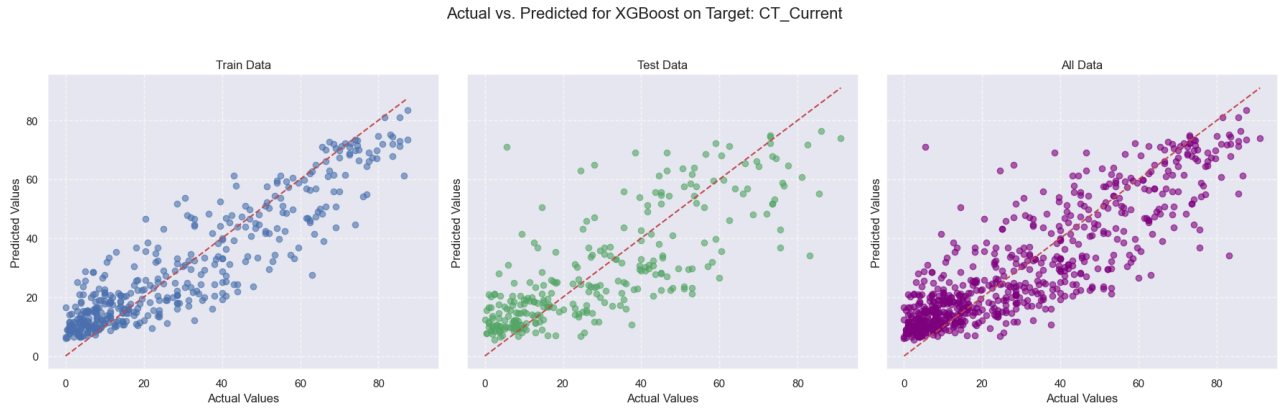


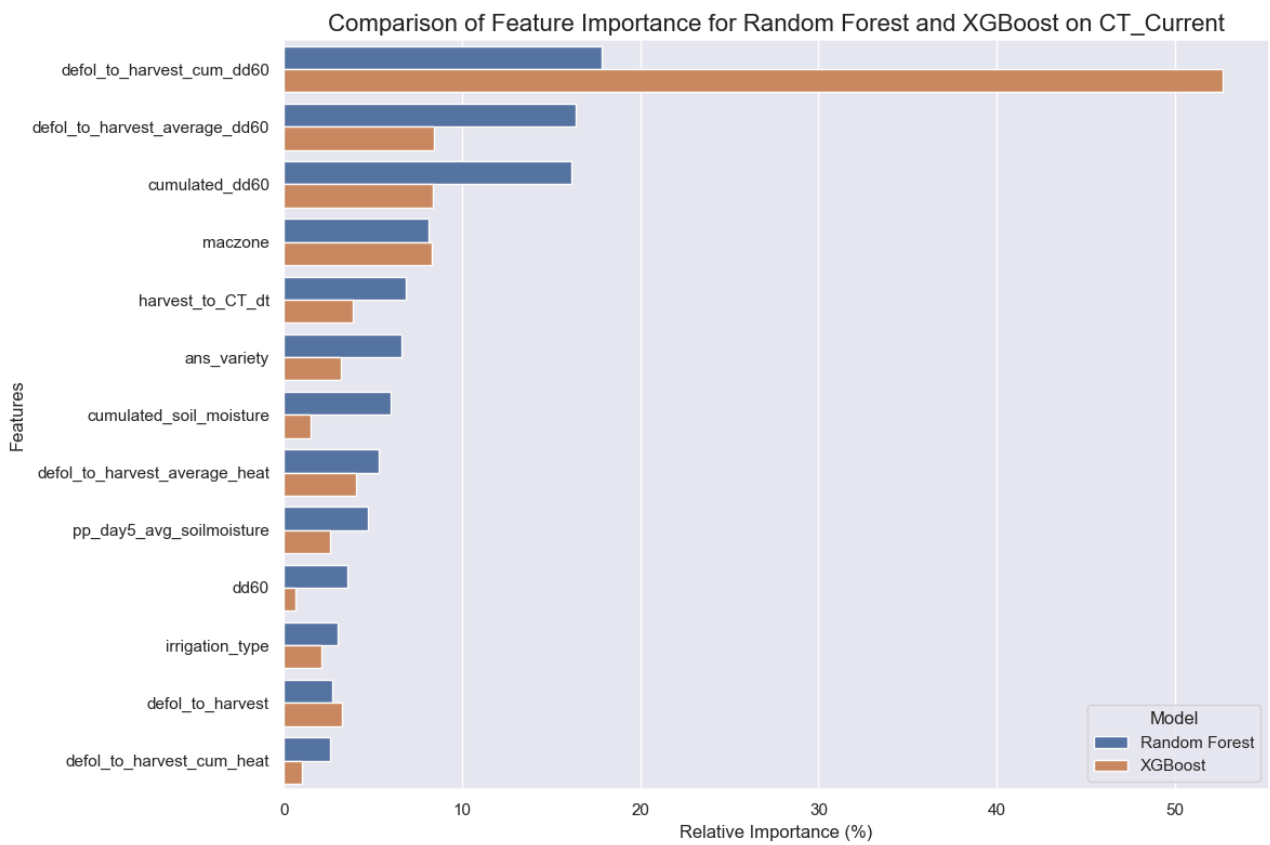

```
LightGBM Best Parameters for CT_Current: {'learning_rate': 0.1, 'max_depth': 4, 'n_estimators': 25, 'verbose': -1}
Best CV R2 Score: 0.3210

LightGBM Results for CT_Current:
Train MSE: 137.3680
Train MAE: 9.3558
Test MSE: 201.7101
Test MAE: 11.0586
Test R2: 0.6104
Cross-Validation R2: 0.6254 (+/- 0.0800)
```

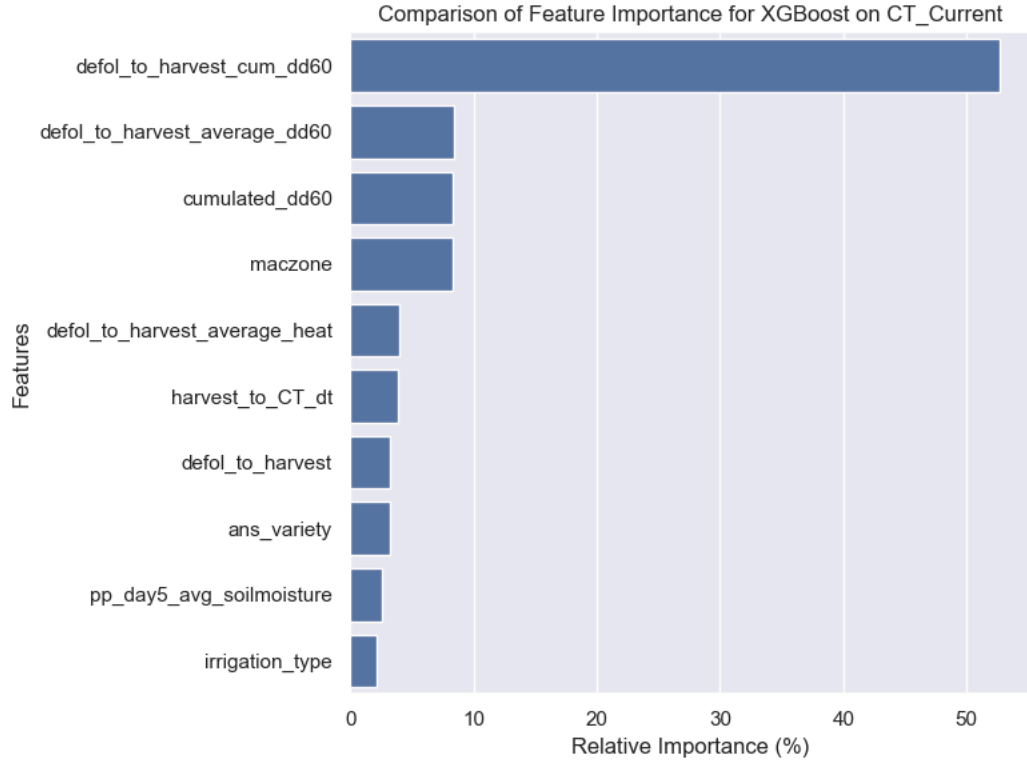



```
XGBoost Best Parameters for CT_Current: {'learning_rate': 0.1, 'max_depth': 3, 'n_estimators': 50}
Best CV R2 Score: 0.3368

Feature Importance for XGBoost (CT_Current):
                        Feature    Importance
8       defol_to_harvest_cum_dd60  25590.207031
6   defol_to_harvest_average_dd60   4082.826416
```

```
2                 cumulated_dd60   4063.359375
10                       maczone   4048.053223
7   defol_to_harvest_average_heat   1961.211914
12               harvest_to_CT_dt   1883.870483
5                 defol_to_harvest   1577.619385
1                      ans_variety   1550.728027
11       pp_day5_avg_soilmoisture   1274.007324
0                 irrigation_type   1036.125488
3         cumulated_soil_moisture    722.838501
9       defol_to_harvest_cum_heat    500.527130
4                            dd60    317.250366

XGBoost Results for CT_Current:
Train MSE: 107.1679
Train MAE: 8.1423
Test MSE: 199.8676
Test MAE: 10.8190
Test R2: 0.6140
Cross-Validation R2: 0.6297 (+/- 0.0719)
```








```
Summary of Results for CT_Current:
               Train MSE Train MAE  Train R2    Test MSE   Test MAE   Test R2  \
Random Forest  63.762797  6.384134  0.894572   204.87653  10.817663  0.604314
LightGBM       137.36802  9.355797   0.77287  201.710113  11.058565  0.610429
XGBoost        107.16792  8.142324  0.822804  199.867606  10.818998  0.613988

              CV R2 Mean CV R2 Std
Random Forest   0.619329   0.04912
LightGBM        0.625406  0.040023
XGBoost         0.629691  0.035967
```

use xgboost model to predict 2025

In [57]:
```
year = 2025
data = rf'C:\Users\EIJEL\OneDrive - Bayer\Personal Data\Erik\2024\Consolidated\{year}cotton_new_latest_withpai
data = pd.read_csv(data)
```

In [58]:
```
data.head(1)
```

Out [58]:

	pa_feature_id	irrigation_type	pa_id	pf_type	variety	harvest_date	cumulated_dd60	cumulated_soil_moisture	dd60	defol_to_harvest	har
0	{00f6ed6f-f180-4879-97eb-0c92cd555059}	FurrowSurge	1783643	Foundation	NaN	2025-10-26	4780.6	6.75	13.7	27.0	8

In [59]:
```
data['CT_Current_DT'] = pd.Timestamp.today().date()
data['CT_Current_DT'] = pd.to_datetime(data['CT_Current_DT'])
data.harvest_date = pd.to_datetime(data.harvest_date)
if 'date' in data.columns:
    data['date'] = pd.to_datetime(data['date'])
else:
    data['date'] = data['harvest_date']
data = data[data.pf_type=="Commercial"]
```

In [60]:
```
data['harvest_to_CT_dt'] = data.CT_Current_DT - data.harvest_date
```

In [61]:
```
data['year'] = data['date'].dt.year
```

In [62]:
```
cols_to_pred = [
    'irrigation_type',
  # 'ans_variety',
      'variety',
  'cumulated_dd60',
  'cumulated_soil_moisture',
  'dd60',
```

```
  'defol_to_harvest',
  'defol_to_harvest_average_dd60',
  'defol_to_harvest_average_heat',
  'defol_to_harvest_cum_dd60',
  'defol_to_harvest_cum_heat',
  'maczone',
  'pp_day5_avg_soilmoisture',
  'harvest_to_CT_dt'
]
```

In [63]:
```
data_pred = data[cols_to_pred]
```

In [68]:
```
data_pred.head()
```

Out [68]:

	irrigation_type	variety	cumulated_dd60	cumulated_soil_moisture	dd60	defol_to_harvest	defol_to_harvest_average_dd60	defol_to_har
1	FurrowSurge	DP2127B3XF	2752.300000	47.220000	15.4	21.0	16.395455	16.684727
2	FurrowSurge	DP2333B3XF	4446.377069	8.819667	7.4	62.0	6.794874	12.479189
5	SubsurfaceDrip	24R6833B3TXF_NPE	2618.820000	35.142667	18.0	59.0	0.603667	10.597857
6	SubsurfaceDrip	DP2239B3XF	3075.600000	29.105000	11.9	3.0	5.550000	14.798000
7	Flood	DP2333B3XF	4486.197874	8.075167	14.8	33.0	4.629349	12.028256

In [65]:
```
data_pred = data_pred.dropna()
```

In [66]:
```
data_pred.shape
```

Out [66]: (591, 13)

In [67]:
```
# def convert_days_to_int(series):
#     """Convert 'n days' strings to integers (e.g., '5 days' -> 5).
#     Handles pandas.NA for missing integer values.
#     """
#     if not isinstance(series, pd.Series):
#         return series # Return as is if not a Series

#     # Apply only if the series contains strings
#     if series.dtype == 'object' or pd.api.types.is_string_dtype(series):
#         return pd.to_numeric(series.str.replace(' days', '', regex=False), errors='coerce').astype('Int64')
#     if pd.api.types.is_timedelta64_dtype(series.dtype):
#         return series.map(lambda x: x.days)
#     return series # Return original series if not string type

data_pred.defol_to_harvest = convert_days_to_int(data_pred.defol_to_harvest)
data_pred.harvest_to_CT_dt = convert_days_to_int(data_pred.harvest_to_CT_dt)
```

In [69]:
```
data_pred = data_pred[data_pred.harvest_to_CT_dt > 0]
```

In [70]:
```
data_pred.shape
```

Out [70]: (263, 13)

In [71]:
```
missing_irr = set(data_pred.irrigation_type.unique()) - set(dftotrain.irrigation_type.unique())
```

In [72]:
```
missing_variety = set(data_pred.variety.unique()) - set(dftotrain.ans_variety.unique())
```

In [73]:
```
data.pf_type.value_counts()
```

Out [73]:
```
pf_type
Commercial    643
Name: count, dtype: int64
```

In [74]:
```
print(missing_irr)
print(missing_variety)
```

```
{'LinearNAC'}
{'24R6535B3TXF_NPE', '24R4035XF_NPE', '24R6012B3TXF_NPE', '24R6833B3TXF', 'ARMOR9413XF', 'DP2525B3XF', '24R6719B3TXF_NPE', '24R6833B3TXF_NPE', 'DP2522
```

In [75]:
```
data_predtr = data_pred[~data_pred.irrigation_type.isin(missing_irr)]
data_predtr = data_predtr[~data_pred.variety.isin(missing_variety)]
```

In [76]:
```
data_predtr.shape
```

Out [76]: (192, 13)

In [77]:
```
loaded_label_encoders = joblib.load('label_encoders.pkl')
# For a specific encoder, e.g., le = loaded_label_encoders['CT_Current_some_column']
```

In [78]:
```
data_predtr.irrigation_type = loaded_label_encoders['CT_Current_irrigation_type'].transform(data_predtr.irriga
data_predtr.variety = loaded_label_encoders['CT_Current_ans_variety'].transform(data_predtr.variety)
```

In [79]:
```
data_predtr.head()
```

Out [79]:

	irrigation_type	variety	cumulated_dd60	cumulated_soil_moisture	dd60	defol_to_harvest	defol_to_harvest_average_dd60	defol_to_harvest_average
1	7	46	2752.3	47.220	15.4	21.0	16.395455	16.684727
6	11	51	3075.6	29.105	11.9	3.0	5.550000	14.798000
10	7	54	2693.5	46.125	15.2	22.0	16.682609	15.329696
11	7	46	2774.1	46.130	10.4	30.0	15.012903	15.945855
17	7	46	2704.1	39.925	13.1	18.0	8.105263	13.419158

In [80]:
```
loaded_xg_model = joblib.load("model_CT_Current_xgboost.pkl")
```

In [81]:
```
# data_2025predtr.defol_to_harvest = convert_days_to_int(data_2025predtr.defol_to_harvest)
# data_2025predtr.harvest_to_CT_dt = convert_days_to_int(data_2025predtr.harvest_to_CT_dt)
```

In [81]:
```
data_predtr = data_predtr.rename(columns = {"variety":"ans_variety"})
```

In [82]:
```
predictions = loaded_xg_model.predict(data_predtr)
```

In [83]:
```
predictions.shape
```

Out [83]: (192,)

In [84]:
```
before_pred = data_pred[data_pred.index.isin(data_predtr.index)]
```

In [86]:
```
# before_pred = before_pred[before_pred.columns[:90]]
```

In [85]:
```
before_pred['initial_ct_pred'] = predictions
```

In [88]:
```
before_pred['initial_ct_pred'] = before_pred['initial_ct_pred'].round(2)
```

In [89]:
```
before_pred.head()
```

Out [89]:

	irrigation_type	variety	cumulated_dd60	cumulated_soil_moisture	dd60	defol_to_harvest	defol_to_harvest_average_dd60	defol_to_harvest_a
1	FurrowSurge	DP2127B3XF	2752.3	47.220	15.4	21.0	16.395455	16.684727
6	SubsurfaceDrip	DP2239B3XF	3075.6	29.105	11.9	3.0	5.550000	14.798000
10	FurrowSurge	DP2333B3XF	2693.5	46.125	15.2	22.0	16.682609	15.329696
11	FurrowSurge	DP2127B3XF	2774.1	46.130	10.4	30.0	15.012903	15.945855
17	FurrowSurge	DP2127B3XF	2704.1	39.925	13.1	18.0	8.105263	13.419158

In [89]:
```
before_pred.to_csv(f'ct_initial_preds_{year}_paid.csv')
```

classification predictions

In [90]:
```
loaded_xg_model_class = joblib.load("XGBoost_CT_Current_best_model_class.pkl")
```

In [91]:
```
data_predtr.head(1)
```

Out [91]:

	irrigation_type	ans_variety	cumulated_dd60	cumulated_soil_moisture	dd60	defol_to_harvest	defol_to_harvest_average_dd60	defol_to_harvest_aver
1	7	46	2752.3	47.22	15.4	21.0	16.395455	16.684727

In [92]:
```
data_predtr['year'] = 2025
data_predtr = data_predtr[['irrigation_type', 'ans_variety', 'cumulated_dd60', 'cumulated_soil_moisture', 'dd6
```

In [93]:
```
# before_pred = data_pred[data_pred.index.isin(data_predtr.index)]
predictions = loaded_xg_model_class.predict(data_predtr)
```

In [94]:
```
before_pred['initial_ct_pred_class'] = predictions
```

In [95]:
```
before_pred.head()
```

Out [95]:

	irrigation_type	variety	cumulated_dd60	cumulated_soil_moisture	dd60	defol_to_harvest	defol_to_harvest_average_dd60	defol_to_harvest_a
1	FurrowSurge	DP2127B3XF	2752.3	47.220	15.4	21.0	16.395455	16.684727
6	SubsurfaceDrip	DP2239B3XF	3075.6	29.105	11.9	3.0	5.550000	14.798000
10	FurrowSurge	DP2333B3XF	2693.5	46.125	15.2	22.0	16.682609	15.329696
11	FurrowSurge	DP2127B3XF	2774.1	46.130	10.4	30.0	15.012903	15.945855
17	FurrowSurge	DP2127B3XF	2704.1	39.925	13.1	18.0	8.105263	13.419158

combine reg and class

In [96]:
```
def category(x):
    if x['initial_ct_pred'] >= 55 and x['initial_ct_pred_class'] > 0:
        return "over threshold"
    if x['initial_ct_pred'] < 60 and x['initial_ct_pred_class'] == 0:
        return "below threshold"
    return "at threshold"
before_pred['category'] = before_pred.apply(lambda x: category(x), axis=1)
```

In [ ]:
```
year
```

In [67]:
```
data.head()
```

Out [67]:

	pa_feature_id	irrigation_type	pa_id	pf_type	variety	harvest_date	cumulated_dd60	cumulated_soil_moisture	dd60	defol_
1	{010c2217-2bfd-4265-bab2-afc8b85072f6}	FurrowSurge	1681976	Commercial	DP2127B3XF	2025-09-22	2752.300000	47.220000	15.4	21.0
2	{014a489b-39eb-4f31-9175-e75552f4bce9}	FurrowSurge	1702347	Commercial	DP2333B3XF	2025-10-28	4339.798333	5.522000	7.4	25.0
3	{018e82ab-de8d-49bd-a37c-d24c88d65c81}	FurrowSurge	1689811	Commercial	DP2333B3XF	2025-10-01	2861.600000	42.635000	17.6	12.0
6	{01de6e7b-ce3f-4b4d-9234-adba990c85ed}	SubsurfaceDrip	1750091	Commercial	24R6833B3TXF_NPE	2025-12-13	2598.336667	34.222333	18.0	52.0
7	{01ec6b3d-1a6d-446d-9c43-2cfe635c94ab}	SubsurfaceDrip	1776019	Commercial	DP2239B3XF	2025-11-12	3067.055000	33.263500	11.9	21.0

In [97]:
```
# before_pred_extra = data[['pa_feature_id', 'pf_feature_id', 'planting_area_id', 'pf_name', 'plantname', 'plan
#     before_pred,
#     left_index=True,
#     right_index=True
# )

before_pred_extra = data[['pa_id']].merge(
    before_pred,
    left_index=True,
    right_index=True
)

# before_pred_extra.to_csv(f'ct_initial_preds_{year}_paid.csv', index=False)

before_pred_extra.to_csv(f'ct_initial_preds_{year}_paid.csv', index=False)
```

In [ ]:

```
###################### try classification #####################################


import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, GridSearchCV, cross_val_score
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from lightgbm import LGBMClassifier
from xgboost import XGBClassifier
# from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score, confusion_r
import warnings
warnings.filterwarnings('ignore')

# Define base features (excluding both target columns and target-specific date features)

base_features = [col for col in cols_to_train if col not in ['CT_Current', 'WG_Current', 'harvest_to_CT_dt', 'l

# Define target-specific feature sets
feature_sets = {
    'CT_Current': base_features + ['harvest_to_CT_dt'],
    # 'WG_Current': base_features + ['harvest_to_WG_dt']
}

# Define binary targets
targets = {
    'CT_Current': dftotrain['CT_Current'] >= 60.0,
    # 'WG_Current': dftotrain['WG_Current'] >= 90.0
}
# Convert boolean targets to integers (0/1)
for target_name in targets:
    targets[target_name] = targets[target_name].astype(int)

# Preprocess features for each target
X_dict = {}
label_encoders = {}
for target_name in targets:
    features = feature_sets[target_name]
    X = dftotrain[features].copy()

    # Identify categorical and numerical columns
    categorical_cols = X.select_dtypes(include=['object']).columns
    numerical_cols = X.select_dtypes(include=['float64', 'int64']).columns

    # Encode categorical variables (including harvest_to_*_dt if categorical)
    for col in categorical_cols:
        le = LabelEncoder()
        X[col] = le.fit_transform(X[col].astype(str))
        label_encoders[f"{target_name}_{col}"] = le

    # Handle missing values (if any)
    X[numerical_cols] = X[numerical_cols].fillna(X[numerical_cols].mean())
    X[categorical_cols] = X[categorical_cols].fillna('missing')

    X_dict[target_name] = X

# Define parameter grids for grid search (adjusted for classification)
param_grids = {
    'Random Forest': {
        'n_estimators': [25, 40, 50, 60, 80],
        'max_features': ['sqrt'],
        'max_depth': [3, 5, 7, 9],
        'min_samples_split': [2, 3, 5],
        'min_samples_leaf': [1, 2, 4],
    },
    'LightGBM': {
        'n_estimators': [25, 50],
        'max_depth': [1, 4, -1],
        'learning_rate': [0.01, 0.1],
        'objective': ['binary']
    },
```

```
    'XGBoost': {
        'n_estimators': [25, 50],
        'max_depth': [3, 4, 5],
        'learning_rate': [0.01, 0.1],
        'objective': ['binary:logistic']
    }
}

# Initialize classification models
model_classes = {
    'Random Forest': RandomForestClassifier(random_state=42),
    'LightGBM': LGBMClassifier(random_state=42),
    'XGBoost': XGBClassifier(random_state=42, eval_metric='logloss')
}

def plot_confusion_matrix(y_true, y_pred, model_name, target_name):
    """Plots a confusion matrix with custom colors and annotations."""
    cm = confusion_matrix(y_true, y_pred)
    tn, fp, fn, tp = cm.ravel()

    plt.figure(figsize=(6, 5))

    # Create an annotated heatmap
    annotations = np.array([[f'TN\n{tn}', f'FP\n{fp}'], [f'FN\n{fn}', f'TP\n{tp}']])

    # Define a custom colormap for green (correct) and red (incorrect)
    cm_plot = sns.heatmap(cm, annot=annotations, fmt='', cmap='viridis', cbar=False, linewidths=0.5, linecolor=

    # Apply color manually to the text annotations
    cm_plot.texts[0].set_color('green')  # TN
    cm_plot.texts[1].set_color('red')    # FP
    cm_plot.texts[2].set_color('red')    # FN
    cm_plot.texts[3].set_color('green')  # TP

    plt.title(f'Confusion Matrix for {model_name} on {target_name}', fontsize=14)
    plt.xlabel('Predicted Label', fontsize=12)
    plt.ylabel('True Label', fontsize=12)
    plt.xticks([0.5, 1.5], ['Negative', 'Positive'])
    plt.yticks([0.5, 1.5], ['Negative', 'Positive'])
    plt.tight_layout()
    plt.savefig(f'{model_name}_{target_name}_confusion_matrix_group.png')
    plt.show()

def plot_roc_curve(y_true, y_proba, model_name, target_name):
    """Plots the ROC-AUC curve."""
    plt.figure(figsize=(6, 5))
    roc_display = RocCurveDisplay.from_predictions(y_true, y_proba)
    roc_display.plot(ax=plt.gca())
    plt.plot([0, 1], [0, 1], color='red', linestyle='--', label='Random Classifier')
    plt.title(f'ROC-AUC Curve for {model_name} on {target_name}\nAUC: {roc_auc_score(y_true, y_proba):.4f}', fo
    plt.xlabel('False Positive Rate', fontsize=12)
    plt.ylabel('True Positive Rate', fontsize=12)
    plt.legend()
    plt.grid(True, linestyle='--', alpha=0.6)
    plt.tight_layout()
    plt.savefig(f'{model_name}_{target_name}_roc_curve_group.png')
    plt.show()

# Function to perform grid search and evaluate classification model
def train_and_evaluate(X, y, model_name, model, param_grid, target_name):
    groups = dftotrain.year
    gkf = GroupKFold(n_splits=len(groups.unique()))
    # Split data
    X_train, X_test, y_train, y_test, groups_train, groups_test = train_test_split(
        X, y, groups, test_size=0.4, random_state=42
    )
    # X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.4, random_state=42)

    # Grid search
    grid_search = GridSearchCV(
        estimator=model,
        param_grid=param_grid,
        cv=gkf,
```

```
    scoring='r2',
    n_jobs=-1
)
grid_search.fit(X_train, y_train, groups=groups_train)

# # Split data
# X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.35, random_state=42, stratify=y)

# # Grid search with F1-score as the primary metric
# grid_search = GridSearchCV(
#     estimator=model,
#     param_grid=param_grid,
#     cv=5,
#     scoring='f1',
#     n_jobs=-1
# )
# grid_search.fit(X_train, y_train)

# Best model
best_model = grid_search.best_estimator_
print(f"\n{model_name} Best Parameters for {target_name}: {grid_search.best_params_}")
print(f"Best CV F1 Score: {grid_search.best_score_:.4f}")

# Predict on train and test sets
y_train_pred = best_model.predict(X_train)
y_test_pred = best_model.predict(X_test)
y_test_proba = best_model.predict_proba(X_test)[:, 1]  # Probabilities for ROC-AUC

# Calculate classification metrics
train_accuracy = accuracy_score(y_train, y_train_pred)
test_accuracy = accuracy_score(y_test, y_test_pred)
test_precision = precision_score(y_test, y_test_pred)
test_recall = recall_score(y_test, y_test_pred)
test_f1 = f1_score(y_test, y_test_pred)
test_roc_auc = roc_auc_score(y_test, y_test_proba)

# Cross-validation on best model
cv_scores = cross_val_score(best_model, X_train, y_train, cv=5, scoring='f1')

# Store results
results = {
    'Train Accuracy': train_accuracy,
    'Test Accuracy': test_accuracy,
    'Test Precision': test_precision,
    'Test Recall': test_recall,
    'Test F1': test_f1,
    'Test ROC-AUC': test_roc_auc,
    'CV F1 Mean': cv_scores.mean(),
    'CV F1 Std': cv_scores.std(),
    'Best Parameters': grid_search.best_params_
}

# Feature importance (for tree-based models)
if model_name in ['Random Forest', 'LightGBM', 'XGBoost']:
    feature_importance = pd.DataFrame({
        'Feature': X.columns,
        'Importance': best_model.feature_importances_
    }).sort_values(by='Importance', ascending=False)
    print(f"\nFeature Importance for {model_name} ({target_name}):")
    print(feature_importance)

cm = confusion_matrix(y_test, y_test_pred)
tn, fp, fn, tp = cm.ravel()

print(f"\nClass Distribution for {model_name} on Test Set:")
print(f"True Positives (TP): {tp}")
print(f"True Negatives (TN): {tn}")
print(f"False Positives (FP): {fp}")
print(f"False Negatives (FN): {fn}")

print(f"\nConfusion Matrix for {model_name} on Test Set:")
print(cm)
plot_confusion_matrix(y_test, y_test_pred, model_name, target_name)
```

```
    plot_roc_curve(y_test, y_test_proba, model_name, target_name)

    return results

# Train and evaluate models for each target
all_results = {}
for target_name, y in targets.items():
    print(f"\n=== Training Models for Target: {target_name} ===")
    X = X_dict[target_name]  # Use target-specific feature set
    all_results[target_name] = {}

    for model_name, model in model_classes.items():
        results = train_and_evaluate(X, y, model_name, model, param_grids[model_name], target_name)
        all_results[target_name][model_name] = results

        print(f"\n{model_name} Results for {target_name}:")
        print(f"Train Accuracy: {results['Train Accuracy']:.4f}")
        print(f"Test Accuracy: {results['Test Accuracy']:.4f}")
        print(f"Test Precision: {results['Test Precision']:.4f}")
        print(f"Test Recall: {results['Test Recall']:.4f}")
        print(f"Test F1: {results['Test F1']:.4f}")
        print(f"Test ROC-AUC: {results['Test ROC-AUC']:.4f}")
        print(f"Cross-Validation F1: {results['CV F1 Mean']:.4f} (+/- {results['CV F1 Std'] * 2:.4f})")

# Summarize results
for target_name in targets:
    print(f"\nSummary of Results for {target_name}:")
    results_df = pd.DataFrame(all_results[target_name]).T
    print(results_df[['Train Accuracy', 'Test Accuracy', 'Test Precision', 'Test Recall', 'Test F1', 'Test ROC-
```

In [ ]:

In [ ]:

```
# import pandas as pd
# import numpy as np
# import torch
# import torch.nn as nn
# import torch.optim as optim
# from sklearn.model_selection import train_test_split, KFold
# from sklearn.preprocessing import LabelEncoder, StandardScaler
# from sklearn.metrics import mean_squared_error, r2_score
# import warnings
# warnings.filterwarnings('ignore')

# # Load your dataframe (replace with your actual data loading)
# # df = pd.read_csv('your_data.csv')
# df = dftotrain
# # Define features (exclude both target columns)
# features = [col for col in dftotrain.columns if col not in ['CT_Initial', 'WG_Initial_7Day_Cnt']]

# # Identify categorical and numerical columns
# categorical_cols = df[features].select_dtypes(include=['object']).columns
# numerical_cols = df[features].select_dtypes(include=['float64']).columns

# # Encode categorical variables and scale numerical variables
# X = dftotrain[features].copy()
# label_encoders = {}
# for col in categorical_cols:
#     le = LabelEncoder()
#     X[col] = le.fit_transform(X[col].astype(str))
#     label_encoders[col] = le

# # Scale numerical features
# scaler = StandardScaler()
# X[numerical_cols] = scaler.fit_transform(X[numerical_cols])

# # Define targets
# targets = ['CT_Initial', 'WG_Initial_7Day_Cnt']

# # Define neural network model
# class RegressionNN(nn.Module):
#     def __init__(self, input_dim, hidden_dim=64, dropout_rate=0.3):
#         super(RegressionNN, self).__init__()
#         self.model = nn.Sequential(
```

```
#             nn.Linear(input_dim, hidden_dim),
#             nn.ReLU(),
#             nn.Dropout(dropout_rate),
#             nn.Linear(hidden_dim, hidden_dim // 2),
#             nn.ReLU(),
#             nn.Dropout(dropout_rate),
#             nn.Linear(hidden_dim // 2, 1)
#         )

#     def forward(self, x):
#         return self.model(x)

# # Function to train and evaluate model
# def train_and_evaluate(X, y, target_name, epochs=100, batch_size=32, device='cpu'):
#     # Split data
#     X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

#     # Convert to PyTorch tensors
#     X_train_tensor = torch.FloatTensor(X_train.values).to(device)
#     y_train_tensor = torch.FloatTensor(y_train.values).reshape(-1, 1).to(device)
#     X_test_tensor = torch.FloatTensor(X_test.values).to(device)
#     y_test_tensor = torch.FloatTensor(y_test.values).reshape(-1, 1).to(device)

#     # Initialize model, loss, and optimizer
#     model = RegressionNN(input_dim=X.shape[1]).to(device)
#     criterion = nn.MSELoss()
#     optimizer = optim.Adam(model.parameters(), lr=0.001, weight_decay=0.01)  # L2 regularization

#     # Train model
#     model.train()
#     for epoch in range(epochs):
#         for i in range(0, len(X_train_tensor), batch_size):
#             batch_X = X_train_tensor[i:i+batch_size]
#             batch_y = y_train_tensor[i:i+batch_size]

#             optimizer.zero_grad()
#             outputs = model(batch_X)
#             loss = criterion(outputs, batch_y)
#             loss.backward()
#             optimizer.step()

#     # Evaluate on train set
#     model.eval()
#     with torch.no_grad():
#         y_train_pred = model(X_train_tensor).cpu().numpy()
#         y_test_pred = model(X_test_tensor).cpu().numpy()

#     train_r2 = r2_score(y_train, y_train_pred)
#     test_mse = mean_squared_error(y_test, y_test_pred)
#     test_r2 = r2_score(y_test, y_test_pred)

#     # Cross-validation
#     kf = KFold(n_splits=5, shuffle=True, random_state=42)
#     cv_scores = []
#     for train_idx, val_idx in kf.split(X_train):
#         X_tr, X_val = X_train.iloc[train_idx], X_train.iloc[val_idx]
#         y_tr, y_val = y_train.iloc[train_idx], y_train.iloc[val_idx]

#         X_tr_tensor = torch.FloatTensor(X_tr.values).to(device)
#         y_tr_tensor = torch.FloatTensor(y_tr.values).reshape(-1, 1).to(device)
#         X_val_tensor = torch.FloatTensor(X_val.values).to(device)

#         model = RegressionNN(input_dim=X.shape[1]).to(device)
#         optimizer = optim.Adam(model.parameters(), lr=0.001, weight_decay=0.01)

#         model.train()
#         for epoch in range(epochs):
#             optimizer.zero_grad()
#             outputs = model(X_tr_tensor)
#             loss = criterion(outputs, y_tr_tensor)
#             loss.backward()
#             optimizer.step()
```

```
#         model.eval()
#         with torch.no_grad():
#             y_val_pred = model(X_val_tensor).cpu().numpy()
#         cv_scores.append(r2_score(y_val, y_val_pred))

#     cv_r2_mean = np.mean(cv_scores)
#     cv_r2_std = np.std(cv_scores)

#     results = {
#         'Train R2': train_r2,
#         'Test MSE': test_mse,
#         'Test R2': test_r2,
#         'CV R2 Mean': cv_r2_mean,
#         'CV R2 Std': cv_r2_std
#     }

#     print(f"\nResults for {target_name}:")
#     print(f"Train R2: {train_r2:.4f}")
#     print(f"Test MSE: {test_mse:.4f}")
#     print(f"Test R2: {test_r2:.4f}")
#     print(f"Cross-Validation R2: {cv_r2_mean:.4f} (+/- {cv_r2_std * 2:.4f})")

#     return results

# # Check for GPU
# device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
# print(f"Using device: {device}")

# # Train and evaluate for each target
# all_results = {}
# for target in targets:
#     print(f"\n=== Training Neural Network for Target: {target} ===")
#     y = dftotrain[target]
#     all_results[target] = train_and_evaluate(X, y, target, device=device)

# # Summarize results
# for target in targets:
#     print(f"\nSummary of Results for {target}:")
#     results_df = pd.DataFrame(all_results[target], index=['Neural Network']).T
#     print(results_df)
```

In [ ]:
```
dftotrain.head()
```

In [ ]:
```
sorted(df_single_batch.columns)
```

In [ ]:
```
cols_to_analyze = [
 'Bulk_Batch',
 'CT_Current',
 'CT_Current_DT',
 'FFA',
 'FUZZY_BRND_DESC',
 'FUZZY_PLNT_CD',
 'FUZZY_VRTY_DESC',
 'LOAD_GR_QTY',
 'Moisture',
 'ORIGIN_REGION',
 'SEASON_YR',
 'Seed_Temperature',
 'WG_Current',
 'WG_Current_DT',
 'ans_variety',
 'catch_rate',
 'centroid',
 'cotton_maczone',
 'country',
 'county',
 'cumulated_dd60',
 'cumulated_soil_moisture',
 'dd60',
 'defol_to_harvest',
 'defol_to_harvest_average_dd60',
 'defol_to_harvest_average_heat',
```

```
 'defol_to_harvest_cum_dd60',
 'defol_to_harvest_cum_heat',
 'field_status',
 'harvest_date',
 'harvest_to_CT_dt',
 'harvest_to_WG_dt',
 'irrigation',
 'irrigation_type',
 'maczone',
 'pa_area',
 'pa_wkt',
 'pa_year',
 'pf_name',
 'pf_year',
 'plant',
 'planted_surface',
 'planting_area_id',
 'planting_date',
 'pp_day10_avg_dd60',
 'pp_day10_avg_soilmoisture',
 'pp_day10_cum_dd60',
 'pp_day10_cum_soilmoisture',
 'pp_day5_avg_dd60',
 'pp_day5_avg_soilmoisture',
 'pp_day5_cum_dd60',
 'pp_day5_cum_soilmoisture',
 'row_spacing',
 'season_length',
 'state',
 'yield',
 'zipcode']
```

In [ ]:
```
df_single_batch[cols_to_analyze].isna().sum()
```

In [ ]:
```
# df_single_batch.defol_to_harvest = convert_days_to_int(df_single_batch.defol_to_harvest)
df_single_batch.zipcode = df_single_batch.zipcode.astype(int)
# df_single_batch.season_length = df_single_batch.season_length.astype(int)
# df_single_batch.harvest_to_WG_dt = df_single_batch.harvest_to_WG_dt.astype(int)
```

In [ ]:
```
df_single_batch[cols_to_analyze].head()
```

In [ ]:
```
df_single_batch.LOAD_GR_QTY.isna().sum()
```

In [ ]:
```

```